



Интро

- ТЕХНИКА

Проверка связи

Если у вас нет звука

убедитесь, что на устройстве и колонках включён звук
обновите страницу или переподключитесь к вебинару
откройте вебинар в другом браузере
перезагрузите устройство и войдите заново

Пожалуйста, отметьтесь в чате



если меня видно и слышно



если есть проблемы со звуком или видео

- 01 / Десктоп

С компьютера

Используйте **Google Chrome** или **Microsoft Edge**.

При проблемах - обновите страницу **F5**

- 02 / Мобильный

С телефона или планшета

Перейдите с мобильного интернета на **WiFi**.

При проблемах - перезапустите приложение Webinar.

- 03 / Общий совет

Если ничего не помогло

Отключитесь и подключитесь заново.

Пишите в чат - организаторы помогут.

- ЗНАКОМСТВО

Перед началом

- КАКИЕ НОВОСТИ?

Что интересного произошло за неделю?

Напишите в чат - событие, новость или вывод из работы, учёбы или личного опыта.



- ПРЕПОДАВАТЕЛЬ КУРСА

Павлин Николай

Преподаватель кафедры бизнес-информатики МТУСИ
с 2021 года

СТО Nova · стартап в сфере мониторинга здоровья · резидент Сколково

Разрабатываю **LLM-приложения** и RAG-платформу **RAGMaster** (грант ФСИ)

Автор RAG-системы для охраны труда - **chat.kiout.ru**, 1000+ пользователей

Соавтор LLM, попавшей в **топ-20** русскоязычных моделей по MERA (02.2025)

7+ лет в коммерческой разработке: Python, FastAPI, LLM-стек, DevOps

Многократный победитель федеральных хакатонов



Telegram



YouTube



LinkedIn

- ДОГОВОРЁННОСТИ

Правила участия

- 1** Продолжительность вебинара - **80 минут**
- 2** Задавайте **вопросы в чате** - ответу в процессе или в конце
- 3** **Запись** будет доступна на платформе на следующий день
- 4** Обсуждение можно **продолжить в чате** после вебинара
- 5** **Воздерживаемся от обсуждения политических тем.** Здесь мы про разработку, ИИ и продукты - давайте сохраним фокус.

Начинаем!





Docker и контейнеризация ИИ-приложений

Модуль 3 Блок 5

Павлин Н.К.

Что уже умеете и откуда стартуем

Из МЗБ4 у вас есть рабочий **FastAPI-сервис** с `/chat`, `/chat/stream`, `/health`, DI, pydantic-settings, unit-тестами.

Сейчас сервис живёт только у вас на ноутбуке. Проблемы:

- У коллеги Python 3.11, у вас 3.13 — разное поведение `TaskGroup` и `typing`
- На CI нет Redis — кеш падает при старте
- Проверяющий диплома не будет ставить ваш `pyproject.toml` руками

Задача блока: упаковать сервис так, чтобы любая машина с Docker поднимала его за одну команду и получала идентичное окружение.

Чего НЕ делаем сегодня:

- Kubernetes, Helm, Swarm
- CI/CD-деплой (GitHub Actions на registry)
- GPU-контейнеры (torch + CUDA — отдельная большая тема)

Словарь занятия

Термин	English	Что это
Образ	Image	Immutable снимок файловой системы со всеми зависимостями
Контейнер	Container	Запущенный процесс из образа в изолированном namespace
Слой	Layer	Один шаг сборки (FROM/COPY/RUN), закешированный отдельно
Тег	Tag	Метка версии образа: <code>python:3.13-slim-bookworm</code>
Registry	Registry	Хранилище образов: Docker Hub, GHCR, ECR
Dockerfile	Dockerfile	Рецепт сборки: инструкции от FROM до CMD
BuildKit	BuildKit	Современный builder: параллельные стадии, cache mounts
Multi-stage	Multi-stage	Сборка в несколько FROM: builder + runtime
ENTRYPOINT / CMD	Entrypoint / CMD	ENTRYPOINT — что запускать, CMD — аргументы по умолчанию
Volume	Volume	Постоянное хранилище вне слоёв образа
Network	Network	Виртуальная сеть между контейнерами
Healthcheck	Healthcheck	Команда проверки «живой ли контейнер»
Compose project	Compose project	Набор сервисов, описанных в <code>compose.yaml</code>
<code>depends_on</code>	depends_on	Порядок старта сервисов + ожидание healthy
env_file	env_file	Внешний файл с переменными окружения

Зачем Docker именно для ИИ-приложений

ИИ-приложения в 2026 — это не один Python-скрипт. Это **композиция разных сервисов**:

- Python 3.13 + FastAPI
- Redis для кеша и rate limit
- Vector DB (Qdrant / pgvector) — появится в M5
- Опционально GPU-драйвер + CUDA для локальных моделей

Что ломается без контейнеров:

- Версии Python: `typing.Self`, `TaskGroup`, behaviour `asyncio.timeout` меняются между 3.11 / 3.12 / 3.13
- Бинарные зависимости: `tiktoken`, `cryptography`, `psycopg` требуют C-компилятора при установке
- Сосуществование: три проекта с разными версиями Redis на одной машине

Что даёт Docker:

- Повторяемость: один `docker build` => тот же образ на ноуте и на сервере
- Изоляция: namespace для PID, network, mount, user
- Деплой: `docker compose up` — единый контракт запуска для dev, CI, prod

Опрос: как вы сейчас используете Docker

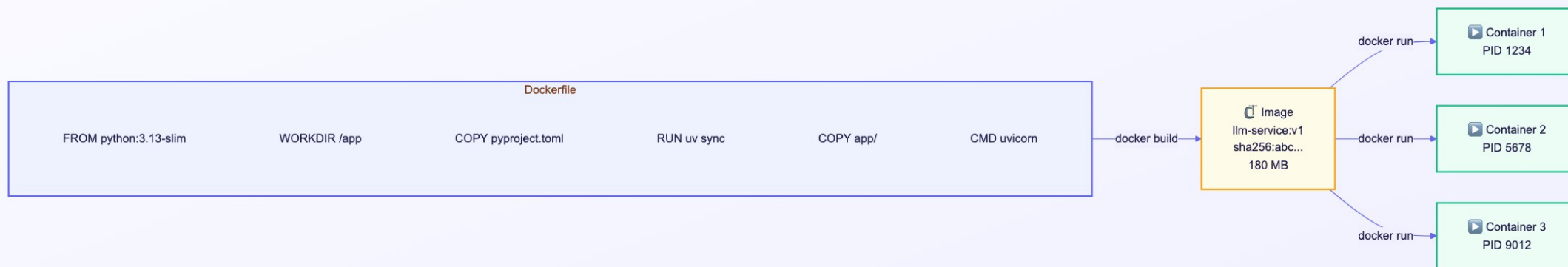
Напишите в чат цифру:

1. Не использую, запускаю всё через `poetry run / uv run`
2. Использую для локальной разработки (Redis, Postgres в контейнере)
3. Использую и для dev, и для prod-депоя
4. Пишу Dockerfile для каждого своего pet-проекта

Image vs Container vs Layer

Три сущности:

- **Image** — Каждая директива Dockerfile — отдельный **слой** с хешем.
- **Container** — запущенная копия образа.
- **Layer** — атомарный закешированный шаг сборки. При изменении одного слоя пересобираются все после него.



Минимальный Dockerfile: как делать НЕ надо

Работает, но плохо:



```
1 FROM python:3.13
2 WORKDIR /app
3 COPY . .
4 RUN pip install -r requirements.txt
5 CMD python -m uvicorn app.main:app
```

Минимальный Dockerfile: как делать НЕ надо

Проблемы:

- `python:3.13` без суффикса = Debian с gcc, perl, man-pages (~1.0 GB)
- `COPY . .` до `pip install` => кеш ломается при любой правке кода
- `COPY . .` тащит `.venv/`, `.git/`, `.env`, `__pycache__` в образ
- Процесс работает от `root` внутри контейнера
- `CMD python -m ...` в shell-форме => PID 1 это sh, сигналы не прокидываются
- Нет `--host 0.0.0.0` => uvicorn слушает только localhost контейнера

Такой образ получается ~1.2 GB, долго собирается, содержит секреты и работает от root.

Правило кеширования слоёв

Ключевое правило: редко меняющееся — в начало Dockerfile, часто меняющееся — в конец.

```
example.dockerfile

1  # 1. Базовый образ — меняется редко
2  FROM python:3.13-slim-bookworm
3
4  # 2. Системные зависимости — почти никогда
5  RUN apt-get update && apt-get install -y --no-install-recommends \
6      build-essential curl \
7      && rm -rf /var/lib/apt/lists/*
8
9  # 3. Python-зависимости — раз в неделю
10 COPY pyproject.toml uv.lock ./
11 RUN uv sync --frozen --no-install-project
12
13 # 4. Код приложения — десятки раз в день
14 COPY app/ ./app/
15 RUN uv sync --frozen --no-dev
```

Правило кеширования слоёв

Эффект: после правки кода пересобираются только последние два слоя (секунды), не `uv sync` (минуты).

Вопрос в чат

Почему `pyproject.toml` и `uv.lock` копируются отдельно от остального кода?

Напишите в чат одним предложением.

Multi-stage build: builder + runtime

```
example.dockerfile

1 # ===== STAGE 1: BUILDER =====
2 FROM python:3.13-slim-bookworm AS builder
3
4 ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy
5
6 COPY --from=ghcr.io/astral-sh/uv:0.11.14 /uv /uvx /bin/
7
8 WORKDIR /app
9 COPY pyproject.toml uv.lock ./
10 RUN --mount=type=cache,target=/root/.cache/uv \
11     uv sync --frozen --no-install-project --no-dev
12
13 COPY app/ ./app/
14 RUN --mount=type=cache,target=/root/.cache/uv \
15     uv sync --frozen --no-dev
16
17 # ===== STAGE 2: RUNTIME =====
18 FROM python:3.13-slim-bookworm
19
20 RUN useradd --create-home --uid 1000 appuser
21 WORKDIR /app
```

```
example.dockerfile

1 COPY --from=builder --chown=appuser:appuser /app /app
2 ENV PATH="/app/.venv/bin:$PATH"
3
4 USER appuser
5 EXPOSE 8000
6
7 ENTRYPOINT ["uvicorn"]
8 CMD ["app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Multi-stage build: builder + runtime

Итог: runtime ~190 MB, без uv, без build tools, non-root.

Сравнение размеров образов

Что оптимизация даёт на практике:

Подход	Размер	Pull time (100 Мбит/с)
<code>python:3.13</code> + <code>pip install</code> + <code>COPY . .</code>	~1150 MB	~90 сек
<code>python:3.13-slim-bookworm</code> + <code>pip install</code>	~220 MB	~18 сек
<code>python:3.13-slim-bookworm</code> + multi-stage + uv	~190 MB	~15 сек
<code>python:3.13-alpine</code> + multi-stage + uv	~130 MB	~10 сек
<code>gcr.io/distroless/python3-debian12</code>	~90 MB	~7 сек

Выбор базового образа: slim vs alpine vs distroless

Правило для ИИ-приложения: `slim-bookworm` на 95% случаев.

Вариант	Плюсы	Минусы	Когда брать
<code>python:3.13</code>	Всё есть из коробки	~1 GB, много CVE	Практически никогда
<code>python:3.13-slim-bookworm</code>	Debian, wheels из PyPI работают	~130 MB базы	По умолчанию для LLM
<code>python:3.13-alpine</code>	~50 MB, быстрый pull	ломает <code>torch</code> , <code>tiktoken</code> wheels	Только чистый Python без C-расширений

BuildKit и cache mounts

BuildKit — современный builder в Docker, включён по умолчанию с Docker 23. Что даёт:

- Параллельное выполнение независимых стадий
- `--mount=type=cache` — персистентный кеш между сборками
- `--mount=type=secret` — передача секретов без записи в layer
- `--platform` — кросс-архитектурная сборка (amd64 + arm64)

Cache mount в действии:

BuildKit и cache mounts

example.dockerfile

```
1 # syntax=docker/dockerfile:1.7
2
3 FROM python:3.13-slim-bookworm
4
5 COPY --from=ghcr.io/astral-sh/uv:0.11.14 /uv /bin/
6
7 COPY pyproject.toml uv.lock ./
8 RUN --mount=type=cache,target=/root/.cache/uv \
9     uv sync --frozen --no-install-project --no-dev
```

BuildKit и cache mounts

Что происходит: uv-кеш (~400 MB) живёт в `/root/.cache/uv` на хосте, не в слое. При повторной сборке пакеты берутся локально — нет сетевых запросов.

uv в Dockerfile: полный production-пример

example.dockerfile

```
1 # syntax=docker/dockerfile:1.7
2 FROM python:3.13-slim-bookworm AS builder
3
4 ENV UV_COMPILE_BYTECODE=1 \
5     UV_LINK_MODE=copy \
6     UV_PYTHON_DOWNLOADS=0
7
8 COPY --from=ghcr.io/astral-sh/uv:0.11.14 /uv /uvx /bin/
9
10 WORKDIR /app
11
12 RUN --mount=type=cache,target=/root/.cache/uv \
13     --mount=type=bind,source=uv.lock,target=uv.lock \
14     --mount=type=bind,source=pyproject.toml,target=pyproject.toml \
15     uv sync --frozen --no-install-project --no-dev
16
17 COPY app/ ./app/
18
19 RUN --mount=type=cache,target=/root/.cache/uv \
20     uv sync --frozen --no-dev
21
22 FROM python:3.13-slim-bookworm
```

example.dockerfile

```
1 RUN useradd --create-home --uid 1000 appuser
2
3 COPY --from=builder --chown=appuser:appuser /app /app
4 ENV PATH="/app/.venv/bin:$PATH"
5
6 WORKDIR /app
7 USER appuser
8 EXPOSE 8000
9
10 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

.dockerignore — обязательный файл

Без .dockerignore:

- `.env` с ключами OpenAI попадает в образ => утечка при push в registry
- `.git` (~50–500 MB) раздувает образ
- `__pycache__` с macOS в Linux-контейнере => мусор
- `.venv` (~300 MB) перезаписывает установленную в образе среду

```
# Git
.git
.gitignore
.github/

# Python
__pycache__/
*.py[cod]
*.py.class
*.so
.Python
.venv/
venv/
env/
.pytest_cache/
.mypy_cache/
.ruff_cache/
.coverage
htmlcov/

# Secrets
.env
.env.*
!.env.example
*.pem
*.key

# IDE
.vscode/
.idea/
*.swp
.DS_Store

# Project
tests/
docs/
*.md
!README.md
data/
notebooks/
```

Non-root user и HEALTHCHECK

example.dockerfile

```
1  # Non-root user — обязательно в production
2  RUN useradd --create-home --shell /bin/bash --uid 1000 appuser
3
4  WORKDIR /app
5  COPY --from=builder --chown=appuser:appuser /app /app
6
7  USER appuser
8
9  # Healthcheck встроен в образ
10 HEALTHCHECK --interval=30s --timeout=5s --start-period=15s --retries=3 \
11     CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health').read()" || exit 1
12
13 EXPOSE 8000
14 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Non-root user и HEALTHCHECK

Зачем non-root:

- Уязвимость в FastAPI (например, RCE через prompt injection + eval) => атакующий получает права процесса
- Root в контейнере = много capabilities, в худшем случае — побег из контейнера
- **USER appuser** = процесс работает с UID 1000, ограниченными правами

Зачем HEALTHCHECK в Dockerfile:

- **docker ps** показывает колонку STATUS: **healthy / unhealthy**
- Orchestrators (k8s, Swarm) используют для рестартов и не-направления трафика
- Compose видит healthcheck и ждёт **service_healthy** при **depends_on**

От одного контейнера к оркестрации

С Dockerfile разобрались — у нас есть один воспроизводимый образ FastAPI. Дальше — Docker Compose, чтобы запускать app вместе с Redis одной командой.

Что было: Dockerfile — multi-stage, uv, non-root, healthcheck, .dockerignore.

Что добавляем: compose.yaml — несколько сервисов в одной декларации, общая сеть, healthcheck-зависимости, named volumes.

Docker Compose v2: compose.yaml

```
example.yaml
1  services:
2    app:
3      build:
4        context: .
5        dockerfile: Dockerfile
6      image: llm-service:dev
7      container_name: llm-service
8      restart: unless-stopped
9      ports:
10       - "8000:8000"
11      env_file:
12       - .env
13      environment:
14        REDIS_URL: redis://redis:6379/0
15        LOG_LEVEL: INFO
16      depends_on:
17        redis:
18          condition: service_healthy
19      healthcheck:
20        test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"]
21        interval: 15s
22        timeout: 5s
23        retries: 3
24        start_period: 15s
```

Docker Compose v2: compose.yaml

```
example.yaml
1
2  redis:
3    image: redis:7.4-alpine
4    container_name: llm-redis
5    restart: unless-stopped
6    volumes:
7      - redis_data:/data
8    healthcheck:
9      test: ["CMD", "redis-cli", "ping"]
10     interval: 10s
11     timeout: 3s
12     retries: 3
13
14 volumes:
15   redis_data:
```

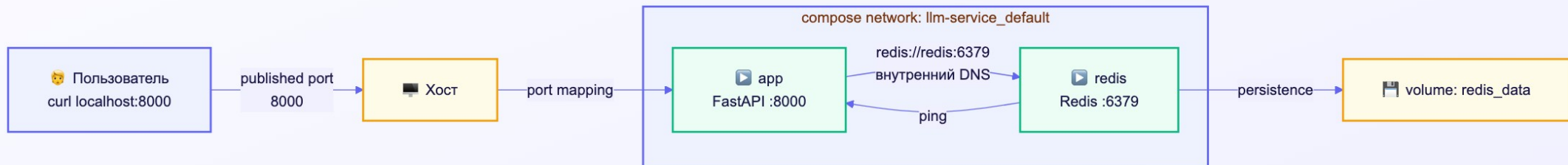
Docker Compose v2: compose.yaml

Запуск: `docker compose up -d --build`

Compose-архитектура: сеть, DNS, изоляция

Что Docker делает автоматически:

- Создаёт bridge-сеть `<project>_default`
- Поднимает DNS-сервер внутри сети: `redis` => IP контейнера
- Изолирует сеть от других compose-проектов
- Пробрасывает только явно указанные `ports` на хост



depends_on: почему нужно service_healthy

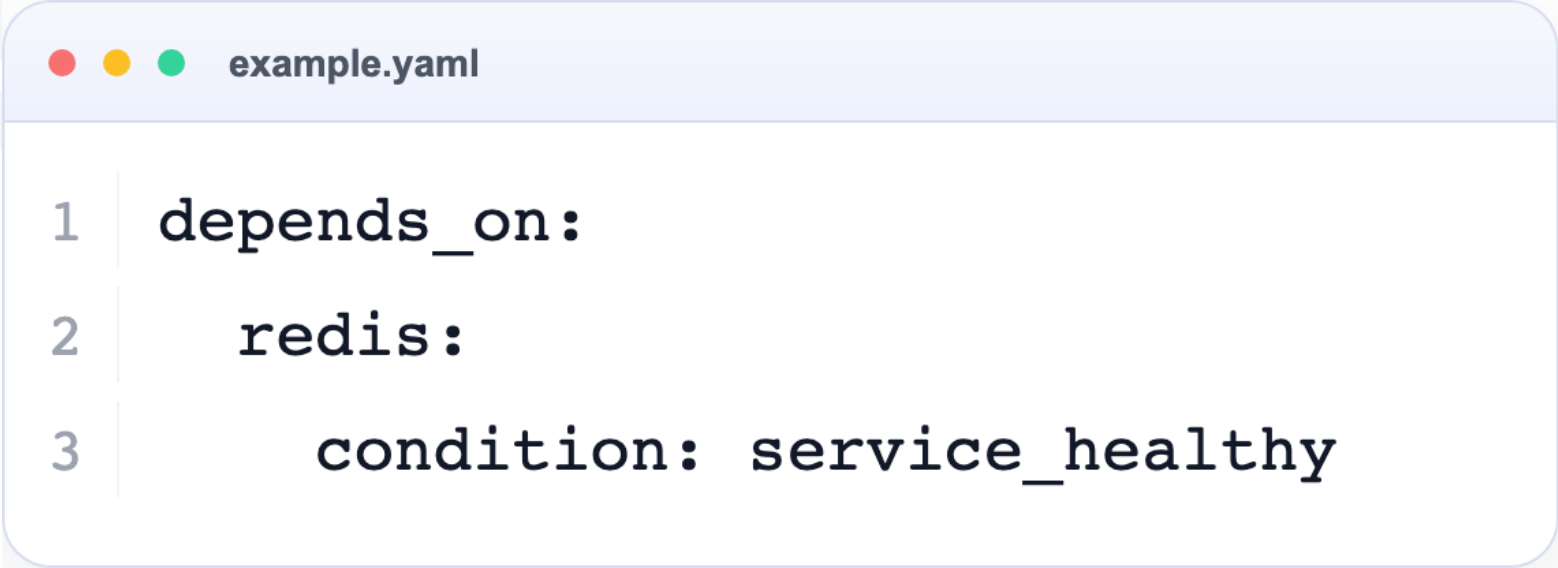
Без `service_healthy`:

```
example.yaml
1 depends_on:
2   - redis # только порядок старта
```

depends_on: почему нужно service_healthy

Что происходит: Docker стартует процесс redis (занимает 200 мс) и **сразу** запускает app. FastAPI в `startup_event` пытается подключиться к Redis — но redis-сервер ещё инициализируется. Ошибка `ConnectionRefusedError`, контейнер падает.

С `service_healthy`:



```
1 depends_on:
2   redis:
3     condition: service_healthy
```

depends_on: почему нужно service_healthy

Docker стартует redis, **ждёт** прохождения healthcheck (`redis-cli ping = PONG`), и только потом запускает app. Гарантированно работоспособный порядок.

Три условия для **condition**:

- `service_started` — процесс запустился (поведение v1, default v2)
- `service_healthy` — healthcheck прошёл **(используйте это)**
- `service_completed_successfully` — контейнер завершился с exit 0 (для миграций БД)

Dev-override: hot reload и bind mount

Создайте второй файл `compose.override.yaml` (автоматически мерджится с основным):

```
example.yaml
1  services:
2    app:
3      command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
4      volumes:
5        - ./app:/app/app:ro
6        - ./tests:/app/tests:ro
7      environment:
8        LOG_LEVEL: DEBUG
9
10   redis:
11     ports:
12       - "6379:6379" # для локального отладчика redis-cli
```

Dev-override: hot reload и bind mount

Что даёт:

- правка `app/main.py` на хосте -> uvicorn сам перезапустится за 300 мс
- цикл «правка -> проверка» сокращается с минут до секунд
- `docker compose up` в dev-директории автоматически применит override

В production `compose.override.yaml` отсутствует — работает только base `compose.yaml`.

Переход: от теории к разбору кода

Дальше — сборка Dockerfile и compose.yaml шаг за шагом по правилам, которые только что разобрали. Готовые файлы лежат в репозитории курса — разбираем построчно.

Что было: теория — слои, кеш, multi-stage, .dockerignore, compose-сеть, depends_on.

Что добавляем: работающий артефакт — `docker compose up` поднимает FastAPI + Redis локально за одну команду.

Разбор: Dockerfile с нуля

Задача: собираем Dockerfile для сервиса из БЗ.4.

Шаг 1. Создаём `.dockerignore` (берём готовый из репозитория).

Шаг 2. Создаём Dockerfile — разбираем построчно:

Разбор: Dockerfile с нуля

```
example.dockerfile

1 # syntax=docker/dockerfile:1.7
2 FROM python:3.13-slim-bookworm AS builder
3 ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy UV_PYTHON_DOWNLOADS=0
4 COPY --from=ghcr.io/astral-sh/uv:0.11.14 /uv /bin/
5 WORKDIR /app
6 COPY pyproject.toml uv.lock ./
7 RUN --mount=type=cache,target=/root/.cache/uv \
8     uv sync --frozen --no-install-project --no-dev
9 COPY app/ ./app/
10 RUN --mount=type=cache,target=/root/.cache/uv uv sync --frozen --no-dev
11
12 FROM python:3.13-slim-bookworm
13 RUN useradd --create-home --uid 1000 appuser
14 COPY --from=builder --chown=appuser:appuser /app /app
15 ENV PATH="/app/.venv/bin:$PATH"
16 WORKDIR /app
17 USER appuser
18 EXPOSE 8000
19 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Разбор: Dockerfile с нуля

Шаг 3. Собираем и проверяем:

```
example.bash
1 docker build -t llm-service:v1 .
2 docker image ls llm-service:v1      # проверяем размер (~190 MB)
3 docker run --rm -p 8000:8000 --env-file .env llm-service:v1
```

Разбор: compose.yaml с Redis

Шаг 1. Создаём `compose.yaml`:

```
example.yaml
1 services:
2   app:
3     build: .
4     ports:
5       - "8000:8000"
6     env_file: .env
7     environment:
8       REDIS_URL: redis://redis:6379/0
9     depends_on:
10      redis:
11        condition: service_healthy
12    healthcheck:
13      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"]
14      interval: 15s
15      timeout: 5s
16      retries: 3
17      start_period: 15s
```

Разбор: compose.yaml с Redis

```
example.yaml
1  redis:
2    image: redis:7.4-alpine
3    volumes:
4      - redis_data:/data
5    healthcheck:
6      test: ["CMD", "redis-cli", "ping"]
7      interval: 10s
8      timeout: 3s
9      retries: 3
10
11 volumes:
12   redis_data:
```

Разбор: compose.yaml с Redis

Шаг 2. Запускаем:

example.bash

```
1 docker compose up -d --build
2 docker compose ps          # оба healthy
3 docker compose logs -f app
```

Разбор: compose.yaml с Redis

Шаг 3. Проверяем:

```
example.bash
1 curl http://localhost:8000/health
2 curl -X POST http://localhost:8000/chat \
3     -H "Content-Type: application/json" \
4     -d '{"messages":[{"role":"user","content":"Hi"}]}'
5 docker compose exec redis redis-cli KEYS '*'
```

Полезные команды и шпаргалка

```
example.bash
1 # Сборка
2 docker build -t llm-service:v1 .
3 docker build --platform linux/amd64 -t llm-service:v1 . # под x86 с Mac
4 docker buildx build --platform linux/amd64,linux/arm64 . # обе архитектуры
5
6 # Compose-жизненный цикл
7 docker compose up -d --build # собрать и запустить в фоне
8 docker compose ps           # статус сервисов
9 docker compose logs -f app   # логи в реальном времени
10 docker compose restart app   # рестарт одного сервиса
11 docker compose stop          # остановить без удаления
12 docker compose down          # остановить и удалить контейнеры
13 docker compose down -v       # + удалить volumes
14
15 # Отладка
16 docker compose exec app bash # зайти в контейнер
17 docker compose exec redis redis-cli # redis CLI внутри контейнера
18 docker compose run --rm app pytest # одноразовый запуск тестов
19
20 # Чистка
21 docker system df             # занятое место
22 docker system prune -f       # удалить висящие образы/контейнеры
23 docker builder prune -f      # чистим builder cache
```

Image size: до и после оптимизации

Замер на нашем FastAPI-сервисе (FastAPI + openai + redis + pydantic-settings):

Суммарно: 1150 MB -> 190 MB. В 6 раз меньше.

Этап оптимизации	Размер	Выигрыш
<code>FROM python:3.13</code> + <code>COPY . .</code> + <code>pip install</code>	1150 MB	baseline
+ slim: <code>python:3.13-slim-bookworm</code>	220 MB	-81%
+ <code>.dockerignore</code> исключает <code>.git</code> , <code>.venv</code> , <code>tests/</code>	210 MB	-5%
+ multi-stage: builder + runtime	200 MB	-5%
+ <code>uv sync --no-dev</code> ВМЕСТО <code>pip install</code>	195 MB	-3%
+ <code>UV_COMPILE_BYTECODE=1</code> , чистка <code>__pycache__</code>	190 MB	-3%
+ <code>distroless/python3-debian12</code> (опц.)	130 MB	-32%

Безопасность: сканирование образов

Инструменты:

- **docker scout** — встроен в Docker Desktop, бесплатный
- **grype** — open-source от Anchore, работает offline
- **trivy** — open-source от Aqua Security, популярный в CI

Безопасность: сканирование образов

example.bash

```
1 # docker scout
2 docker scout cves llm-service:v1
3 docker scout recommendations llm-service:v1
4
5 # gype
6 gype llm-service:v1
7
8 # trivy
9 trivy image llm-service:v1
```

Безопасность: сканирование образов

Что показывают:

- CVE в базовом образе (`openssl`, `libc`)
- CVE в Python-зависимостях (из `pip freeze`)
- Рекомендации: «обновись на `python:3.13.1` — закрывает 3 High»

Практика безопасности:

- Сканирование на каждый build в CI
- Блокировка merge, если появились High/Critical CVE
- Регулярная (раз в неделю) пересборка образа — базовый образ обновляется

Антипаттерны и частые ошибки

1. `uvicorn` слушает только `localhost`

Забыли `--host 0.0.0.0` => контейнер недоступен снаружи. Симптом: `curl localhost:8000` с хоста — connection refused.

2. Redis подключается к `localhost`

`REDIS_URL=redis://localhost:6379` в контейнере — это сам контейнер, не Redis. Правильно: `redis://redis:6379` (имя сервиса).

3. `latest` в `image: redis:latest`

Сегодня 7.4, завтра 8.0 с breaking changes. Фиксируйте теги: `redis:7.4-alpine`.

4. `.env` в образе

Нет `.dockerignore` => `COPY . .` тащит секреты. Ключи утекают при первом `docker push`.

5. CMD в shell-форме

`CMD uvicorn ...` вместо `CMD ["uvicorn", ...]` => PID 1 это sh, сигналы не пробрасываются, `docker stop` работает через SIGKILL.

6. Один `compose`-файл на `dev` и `prod`

`--reload` в проде, bind mounts с хоста, порты БД наружу. Разделяйте: `compose.yaml` + `compose.override.yaml`.

7. Нет `healthcheck`

`depends_on` только порядок старта, а не готовность. App падает, потому что Redis ещё инициализируется.

Lifecycle образа: от источника до прода

Ключевые переходы:

- Source -> Image: детерминистичная сборка на BuildKit
- Image -> Registry: публикация с семантическим тегом
- Registry -> Prod: декларативный `docker compose pull && up`
- Scan -> Source: регулярное пересборка для закрытия CVE



Готовые шаблоны для старта

Не каждый проект писать с нуля — есть удобные стартовые шаблоны.

- [fastapi-docker-boilerplate](#) — каркас FastAPI + Docker под production: multi-stage [Dockerfile](#), [compose.yaml](#), разбиение [app/](#) на слои (core, deps, routers, services, schemas), готовые healthcheck-и и [.env.example](#).

Что брать:

- Структуру каталогов и устройство [lifespan](#) — экономит ~2 часа на старте проекта.
- Шаблон [Dockerfile](#) и [compose.yaml](#) — копировать как референс, адаптировать под свой [pyproject.toml](#).
- [.dockerignore](#) и схема секретов — выверенные значения, не нужно изобретать.

Что НЕ брать:

- Слепо копировать всё — потеряете понимание. Сначала собрать самостоятельно (на этом занятии), потом сверяться с шаблоном.

GPU в Docker: если в проде локальная модель

В этом курсе мы работаем с cloud-моделями (OpenAI), GPU не нужен. Но если в дипломе или после курса потребуется **локальная LLM в продакшне** (Ollama, vLLM) или **локальные эмбединги** (M5: sentence-transformers, BGE-M3) — понадобится проброс GPU в контейнер.

Что нужно для GPU в Docker:

1. **nvidia-container-toolkit** на хосте — позволяет Docker видеть GPU через CDI/runtime.
2. **Base image с CUDA** — `nvidia/cuda:12.4.0-runtime-ubuntu22.04` или сборки от vendor-a (PyTorch, TensorRT).
3. **Флаг `--gpus all`** в `docker run` или `deploy.resources.reservations.devices` в compose.

Готовый шаблон: [fastapi-gpu-docker-deploy](#) — FastAPI с пробросом GPU, рабочий **Dockerfile** на базе CUDA, `compose.yaml` с GPU-секцией, healthcheck. Можно взять как стартовую точку.

Когда это пригодится в курсе:

- М1Б3 — Ollama локально (без Docker).
- М5Б1 — локальные эмбединги в RAG-пайплайне.
- Диплом — если выбрали локальную модель как часть стека.

Итоги блока

Что умеете после занятия:

- Пишете multi-stage Dockerfile с uv, non-root user, BuildKit cache mounts
- Настраиваете `.dockerignore` — секреты и мусор не попадают в образ
- Собираете `compose.yaml` с app + Redis, `depends_on: service_healthy`, named volumes
- Различаете dev-override (`--reload`, bind mounts) и production-compose
- Выбираете базовый образ осознанно: `slim-bookworm` для LLM
- Знаете, что проверять через `docker scout/grype`

Чего хватит для диплома:

- Один `Dockerfile` + один `compose.yaml` + один `.dockerignore`
- Команда `docker compose up -d --build` поднимает весь стек
- `/health` возвращает `ok` через 15 секунд после старта

Домашнее задание (ДЗ-самост)

Задача: контейнеризировать FastAPI-сервис из МЗБ4.

Минимум (обязательно):

1. `Dockerfile` — multi-stage, `python:3.13-slim-bookworm`, `uv` или `poetry`, non-root user
2. `.dockerignore` — исключает `.env`, `.git`, `__pycache__`, `.venv`, `tests/`
3. `compose.yaml` — сервисы `app` + `redis`, `depends_on: service_healthy`, named volume для Redis
4. HEALTHCHECK в `app` (через `/health`) и `redis` (через `redis-cli ping`)
5. Команда `docker compose up -d --build` поднимает стек без ошибок

Дополнительно (бонус):

- `compose.override.yaml` с hot reload и bind mount для dev
- Прогон `docker scout cves` и обработка High-уязвимостей
- Замер размера образа до/после multi-stage — скриншот

Сдача: в репозиторий дипломного проекта, PR с тегом `block-3-5`. Без формального ревью.

Связь с дипломом: это чекпоинт 3 модуля. Без рабочего `docker compose up` диплом не принимается.

Референсы — Docker, Python, образы

Официальная документация:

- [Docker docs — Build, best practices](#)
- [Docker docs — Multi-stage builds](#)
- [Docker docs — Compose specification](#)
- [Docker docs — Dockerfile reference](#)

Python и uv:

- [uv docs — Using uv in Docker](#)
- [astral-sh/uv-docker-example](#)
- [Hynek — Production-ready Python Docker Containers with uv](#)

Образы:

- [Python official images](#)
- [Google Distroless](#)
- [Chainguard Images](#)

Референсы — безопасность, compose, деплой

Безопасность и линтеры:

- [Docker Scout](#)
- [Anchore Gype](#)
- [Aqua Trivy](#)
- [OWASP Docker Security Cheat Sheet](#)
- [Hadolint — Dockerfile linter](#)
- [Compose depends_on with service_healthy](#)

Деплой и эксплуатация (для тех, кто пойдёт дальше Compose):

- [Kamal](#) — zero-downtime деплой Docker-приложений на VPS без Kubernetes
- [Databasus](#) — open-source бэкапы PostgreSQL/MySQL/MongoDB в S3 по cron
- [PGTune](#) — генератор `postgresql.conf` под характеристики железа